딜러와 플레이어의 대결

-블랙잭-

상명대학교 휴먼지능정보공학전공 기깔나는팀

CONTENTS

01. 블랙잭 게임설명

02. 게임 구현과정

03. 코드 설명

04. 게임 실행

블랙잭, 어떻게 하지?

주요 용어:

1. ***Hit(히트)*** - 카드 1장 추가, ***Stay(스테이)***- 카드 더 받지 않음.
2. ***Burst(버스트)*** : 플레이어 또는 딜러의 카드 합이 21을 초과한 경우 => 패
3. ***push(푸시)*** : 플레이어와 딜러 카드 합이 같은 경우 => 무승부

기본 규칙:

1. 블랙잭은 각 플레이어(1명 이상)와 딜러의 경쟁
2. 각자 최종 카드들의 합이 ***21과 같거나 가장 가까운*** 사람이 승리 (초과(버스트)시, 패배)
   - 무승부(푸시)시, 배팅금액은 본인에게 그대로 돌아감.
3. 카드 계산: 숫자 2~9는 고유 숫자 그대로, 10/J/Q/K는 모두 값 10으로 통일
   - A= 1 or 11. BUt, 우리 프로그램에서는 11이 우선 적용된 경우의 합이 21을 초과할시, 1 적용으로 바뀜.
4. 처음에 카드를 2장씩 받으며, 딜러는 두 카드 중 하나만 플레이어들에게 공개
5. 이후, 플레이어가 히트 혹은 스테이를 선택하면
   딜러도 히트 혹은 스테이를 선택하되 플레이어는 그 여부를 알 수 없음.
6. 마지막으로, 딜러의 카드가 공개!! 바로, 승패여부가 결정됨~!!

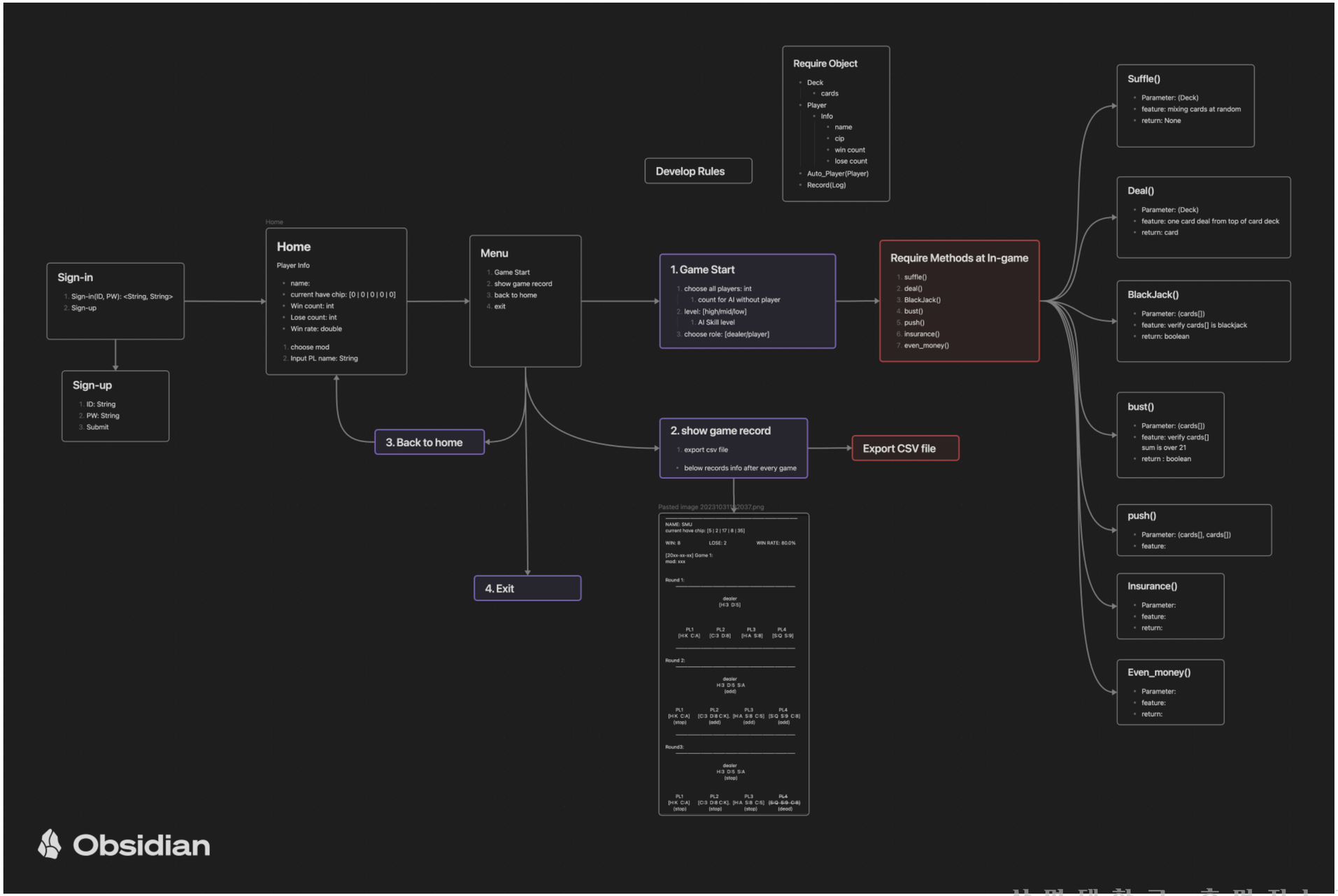
CONTENTS

01. 블랙잭 게임설명

02. 게임 구현과정

03. 코드 설명

04. 게임 실행

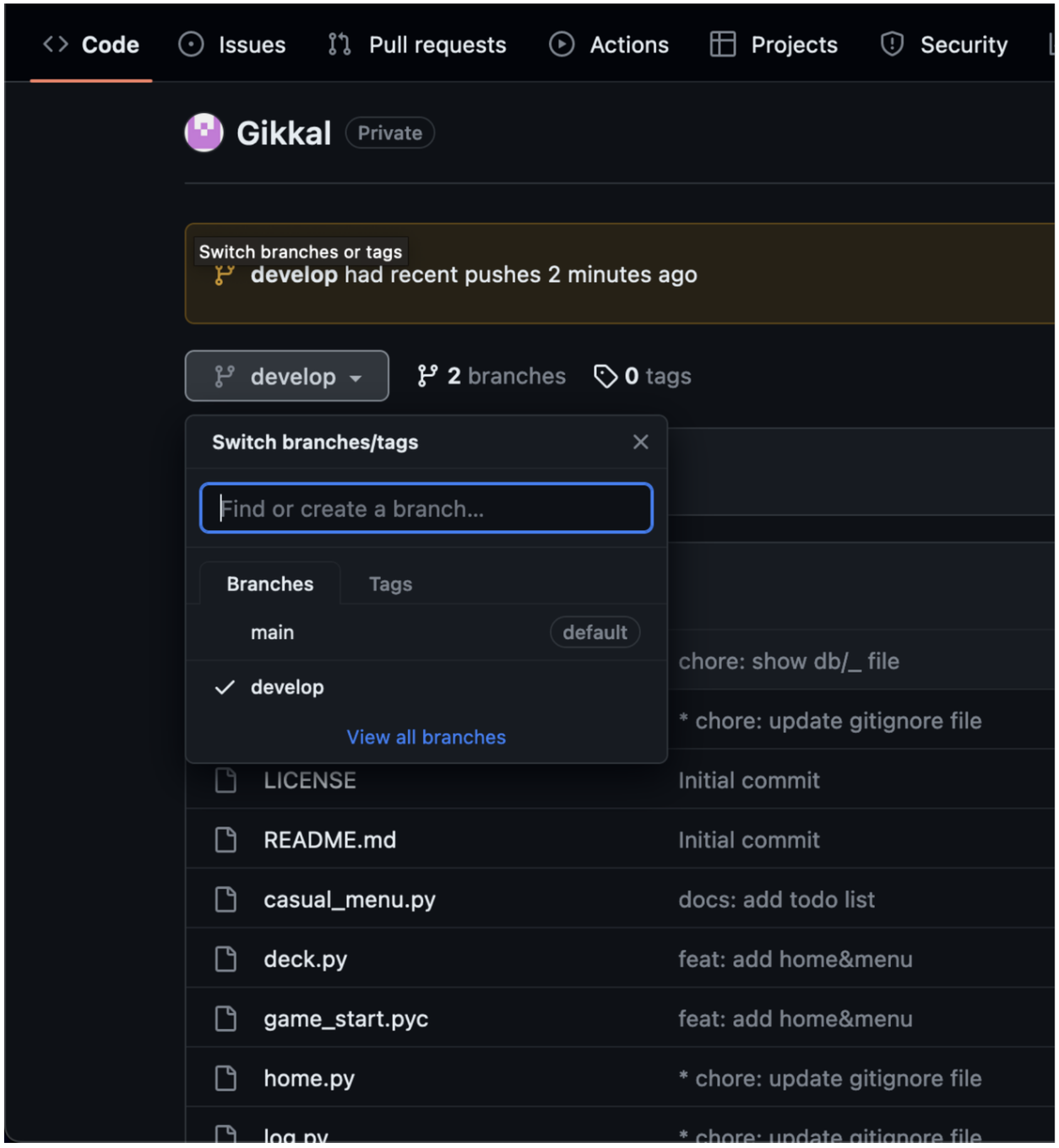


초반 회의 내용

옵시디언 이용

개발자 플랫폼

GitHub 사용



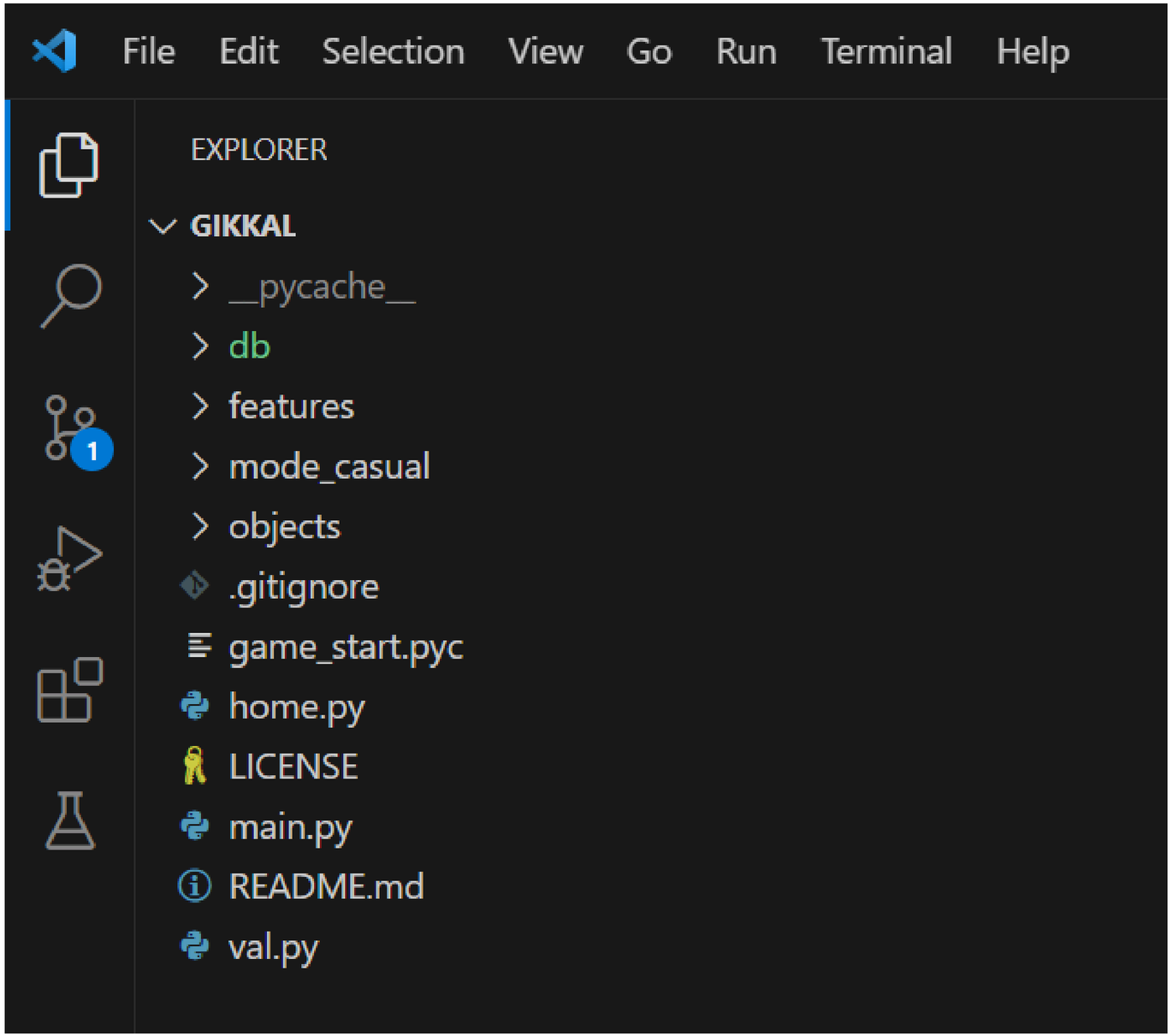
CONTENTS

01. 블랙잭 게임설명

02. 게임 구현과정

03. 코드 설명

04. 게임 실행


File Edit Selection View Go Run Terminal Help
EXPLORER
GIKKAL
__pycache__
db
features
mode_casual
objects
.gitignore
game_start.pyc
home.py
LICENSE
main.py
README.md
val.py

전체적인 구조

main.py > ...

```python
import home
import features.game as game
from objects.card import Card
from objects.player import Player

name = input("Please input your name : ")
pl = home.init_db(name)
home.lobby(pl)
```

부분 설명

main.py

- 모듈을 가져오고 게임이나 애플리케이션과 관련된 일부 코드 포함
- 플레이어 정보 설정, 데이터베이스 초기화, 애플리케이션 내 로비 탐색

실행 버튼을 누르면~! =>

```
Please input your name :
```

```
Please input your name : king
Sign-up: king

Player Info
-------------------------------------
name : king
current chip : [10 | 0 | 0 | 0 | 0 | 0]
win count : 0
lose count : 0
win rate : 0.0%
-------------------------------------



Choice game mode
-------------------------------------
1. Casual
2. !Basic! (Not yet...)
3. show money($)
-------------------------------------
```

home.py > init_db

```python
import os
import mode_casual.casual_menu as casual_menu
import features.logs as logs
import features.calc_chip as calc
from objects.player import Player
from val import CLI_I

def init_db(name):
    file_path = logs.get_db_file_path(name)
    if os.path.exists(file_path):
        print(f"Sign-in: {name}")
        pl = logs.player_txt_to_object(file_path)
        return pl

    print(f"Sign-up: {name}")
    f = open(file_path, "w")
    pl = Player(name, [10, 0, 0, 0, 0, 0])

    f.write(logs.player_object_to_txt(pl))
    f.close()
    return pl


def choose_mod(pl):
    mode = input(CLI_I.DB_MENU)
    if mode == "1":
        casual_menu.choice_menu(pl)
    elif mode == "2":
        print("Implementing...")
        choose_mod(pl)
    elif mode == "3":
        print('current money: ', (int)(calc.calc_money(pl.get_curr_chip())),'$')
        choose_mod(pl)

def lobby(pl):
    print(logs.player_object_to_txt(pl))
    choose_mod(pl)
```

```
Choice game mode
-------------------------------
1. Casual
2. !Basic! (Not yet...)
3. show money($)
-------------------------------
1

choice :
-------------------------------
1. Game Start
2. Show game record
3. Back to home
4. Explain rule
5. Exit
-------------------------------
```

mode_casual > casual_menu.py > ...

```python
import home
import features.logs as logs
from val import (CLI_I, FILE)
import objects.result as game

def choice_menu(pl):
    menu = input(CLI_I.CHOICE_MENU)
    if menu == "1":
        game.game_start(pl)
    elif menu == "2":
        num = input(CLI_I.EXPORT_FILE_MENU)
        if num == '1':
            export_file(FILE.CSV, pl.name)
        elif num == '2':
            export_file(FILE.ZIP, pl.name)
        elif num == "3":
            home.lobby(pl)
    elif menu == "3":
        logs.back_to_home(pl)
    elif menu == "4":
        print(CLI_I.EXPLAIN_RULE)
        choice_menu(pl)
    elif menu == "5":
        exit(0)

def export_file(category,  name):
    message = ""
    if category == FILE.CSV:
        message = logs.export_txt_to_csv(name, logs.get_db_file_path(name))
    elif category == FILE.ZIP:
        message = logs.download_all_log(name, logs.get_db_file_path(name))
    print(f"Successfully export {category} file!!")
    print(f"{category} file dir: ", message)
```

```
choice :
----------------------------------------
1. Game Start
2. Show game record
3. Back to home
4. Explain rule
5. Exit
----------------------------------------
1
```

```
----------------------------------------
1. Select Player Count and Game Start
2. Back to home
1
```

```
Player Count(1~3): 2
```

```
Input bet price: (1~1000): 500
```

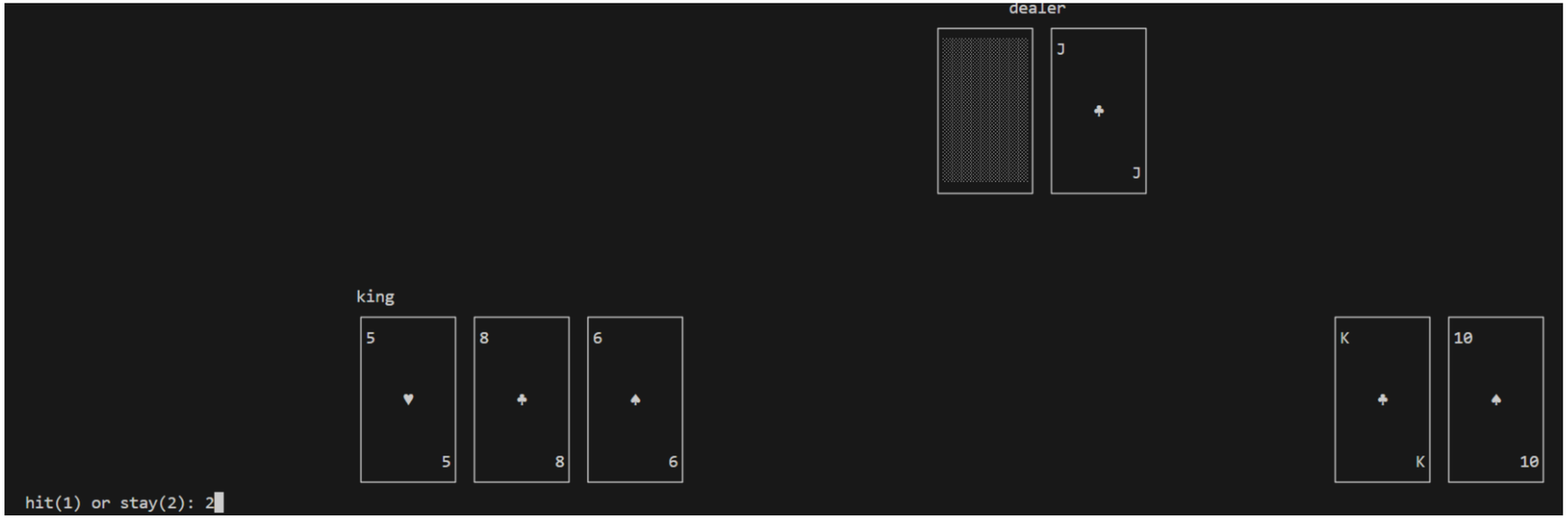
```
Input bet price: (1~1000): 500

                                                dealer
                                                J
                                                ♣
                                                J

king
5                8
♥                ♣
5                8

                                                K                10
                                                ♣                ♠
                                                K                10

hit(1) or stay(2):
```

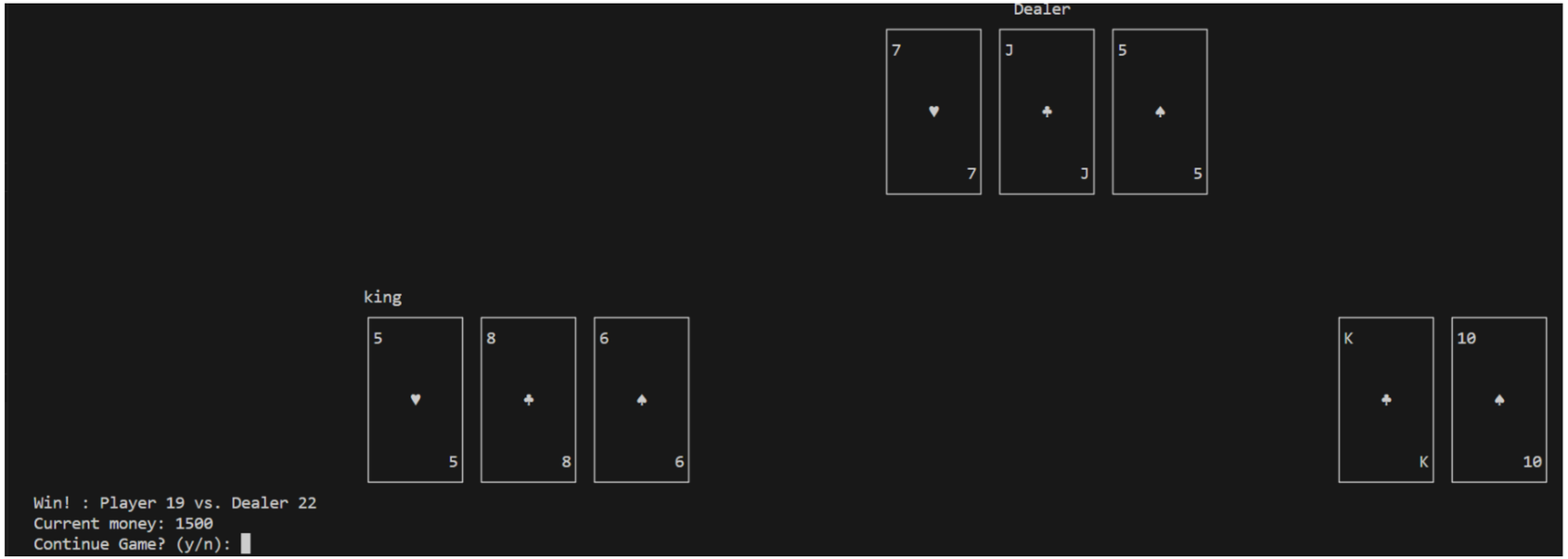
objects > card.py > Card

```python
class Card(object):

    card_values = {
        'Ace': 11,  # value of the ace is high until it needs to be low
        '2': 2,
        '3': 3,
        '4': 4,
        '5': 5,
        '6': 6,
        '7': 7,
        '8': 8,
        '9': 9,
        '10': 10,
        'Jack': 10,
        'Queen': 10,
        'King': 10
    }

    def __init__(self, suit, rank):
        """
        :param suit: The face of the card, e.g. Spade or Diamond
        :param rank: The value of the card, e.g 3 or King
        """
        self.suit = suit.capitalize()
        self.rank = rank
        self.points = self.card_values[rank]
        self.name = f'{rank} of {suit}'
```

```python
def ascii_version_of_card(*cards, return_string=True):
    """
    Instead of a boring text version of the card we render an ASCII
    :param cards: One or more card objects
    :param return_string: By default we return the string version o
    keep it as a list so that the dealer can add a hidden card in f
    """
    # we will use this to prints the appropriate icons for each car
    suits_name = ['Spades', 'Diamonds', 'Hearts', 'Clubs']
    suits_symbols = ['♠', '♦', '♥', '♣']

    # create an empty list of list, each sublist is a line
    lines = [[] for i in range(9)]

    for index, card in enumerate(cards):
        # "King" should be "K" and "10" should still be "10"
        if card.rank == '10':  # ten is the only one who's rank is
            rank = card.rank
            space = ''  # if we write "10" on the card that line wi
        else:
            rank = card.rank[0]  # some have a rank of 'King' this
            space = ' '  # no "10", we use a blank space to will th
        # get the cards suit in two steps
        suit = suits_name.index(card.suit)
        suit = suits_symbols[suit]

        # add the individual card on a line by line basis
        lines[0].append('┌─────────┐')
        lines[1].append('│{}{}      │'.format(rank, space))  # use
        lines[2].append('│         │')
        lines[3].append('│         │')
        lines[4].append('│    {}   │'.format(suit))
        lines[5].append('│         │')
        lines[6].append('│         │')
        lines[7].append('│      {}{}│'.format(space, rank))
        lines[8].append('└─────────┘')
```

```python
def ascii_version_of_hidden_card(*cards):

    """
    Essentially the dealers method of print ascii cards. This method hides the first card, shows it flipped over
    :param cards: A list of card objects, the first will be hidden
    :return: A string, the nice ascii version of cards
    """
    # a flipper over card. # This is a list of lists instead of a list of string becuase appending to a list is better then adding a string
    print('\t'*13,  end='')
    print('dealer')
    lines = [['\t'*12 + '┌─────────┐'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'],
             ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '│▒▒▒▒▒▒▒▒▒│'], ['\t'*12 + '└─────────┘']]
```

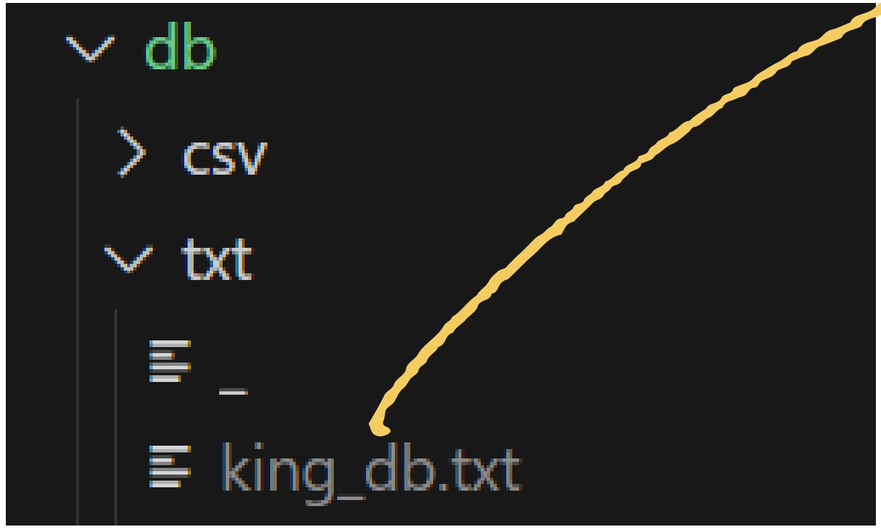




- 딜러의 히트 또는 스테이 여부는 알 수 없음!
- 마지막에는 모든 카드가 공개!!
- 딜러의 카드 버스트 되었으므로 플레이어 승리!
- 현재 보유 머니: 1500

```
Continue Game? (y/n): n
Bye.
```

게임 종료 후,
다시 로그인을 하면?



저장된 플레이어의 이전 기록을 볼 수 있다!!!!

```
Please input your name : king
Sign-in: king

Player Info
---------------------------------------
name : king
current chip : [15 | 0 | 0 | 0 | 0 | 0]
win count : 1
lose count : 0
win rate : 100.0%
---------------------------------------


Choice game mode
----------------------------------------
1. Casual
2. !Basic! (Not yet...)
3. show money($)
----------------------------------------
```

CONTENTS

01. 블랙잭 게임설명

02. 게임 구현과정

03. 코드 설명

04. 게임 실행

File Edit Selection View Go Run Terminal Help

Gikkal

EXPLORER

GIKKAL

- zip
- _
- s_db.txt U
- features
  - __pycache__
  - calc_chip.py
  - game.py
  - logs.py
- mode_casual
  - __pycache__
  - casual_menu.py
- objects
  - __pycache__
  - action.py
  - betting.py
  - card.py
  - deck.py
  - player.py
  - playgame.py
  - result.py
- .gitignore
- game_start.pyc
- home.py
- LICENSE
- main.py
- README.md
- val.py

OUTLINE

TIMELINE

VS CODE PETS

Welcome | main.py

main.py > ...

```python
import home
import features.game as game
from objects.card import Card
from objects.player import Player

name = input("Please input your name : ")
pl = home.init_db(name)
home.lobby(pl)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Filter (e.g. text, **/*.ts, !**/node_modules/**)

No problems have been detected in the workspace.

바탕 화면 보기

develop* 0 0 0

Ln 8, Col 15 Spaces: 4 UTF-8 CRLF Python 3.11.2 64-bit

감사합니다

기깔나는팀

팀장 이선역　　팀원 강동현

팀원 송예린　　팀원 유세리